



Sesión 2: Implementación Avanzada de Autenticación

Curso IV — Seguridad Aplicada a Entornos Java & .NET

DURACIÓN: 3 H 20 MIN

NIVEL SENIOR

Objetivos de la sesión

Al finalizar esta sesión, los participantes serán capaces de configurar frameworks de autenticación de forma avanzada, implementar los flujos OAuth2 más relevantes y gestionar el ciclo de vida completo de los tokens en entornos productivos.

Spring Security / ASP.NET Identity

Configuración avanzada e integración con proveedores externos

Flujos OAuth2

Authorization Code, Client Credentials y PKCE en profundidad

Gestión de tokens

Firma, validación, expiración y buenas prácticas de almacenamiento

Taller práctico

Implementación real con JWT y control de acceso

Agenda de la sesión

01

Spring Security y ASP.NET Identity

Configuración avanzada, integración con IdPs externos, gestión de usuarios y roles — **60 min**

03

Gestión de tokens JWT

Firma, validación, expiración, renovación y almacenamiento seguro — **40 min**

02

Flujos OAuth2

Authorization Code, Client Credentials y PKCE con análisis de casos de uso — **50 min**

04

Taller práctico

Implementación guiada de autenticación basada en tokens y control de acceso — **50 min**



BLOQUE 1

Spring Security y ASP.NET Identity

Configuración avanzada, integración con proveedores externos y gestión de usuarios y roles

¿Por qué frameworks dedicados de seguridad?

Implementar autenticación y autorización desde cero es una fuente habitual de vulnerabilidades críticas. Los frameworks especializados proporcionan abstracciones probadas, integración con estándares y un modelo de extensión que reduce la superficie de ataque.

Spring Security

- Ecosistema Java/Kotlin maduro y ampliamente adoptado
- Integración nativa con Spring Boot y Spring Cloud
- Soporte completo de OAuth2, OIDC, SAML y LDAP
- Filtros, interceptores y DSL de configuración declarativa

ASP.NET Core Identity

- Framework de identidad nativo del ecosistema .NET
- Gestión de usuarios, roles, claims y políticas integrada
- Extensible mediante stores personalizados (EF Core, Dapper...)
- Compatible con OpenIddict, IdentityServer y Duende

Spring Security: Arquitectura de filtros

Spring Security se integra en el pipeline HTTP de Spring mediante una cadena de filtros (**SecurityFilterChain**). Cada solicitud entrante atraviesa estos filtros secuencialmente antes de llegar al controlador. Comprender este modelo es fundamental para configuraciones avanzadas.



El orden de los filtros es determinante. Los filtros de autenticación deben ejecutarse antes que los de autorización. Spring Security garantiza este orden por defecto, pero puede sobrescribirse con `@Order` o mediante la configuración del `SecurityFilterChain`.

Spring Security: Configuración avanzada con SecurityFilterChain

Desde Spring Security 5.7, la configuración basada en herencia de `WebSecurityConfigurerAdapter` está deprecada. El enfoque moderno usa beans de tipo `SecurityFilterChain` para una configuración más modular y testeable.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt.jwtAuthenticationConverter(
                    jwtAuthenticationConverter())));
        return http.build();
    }
}
```

 El uso de `SessionCreationPolicy.STATELESS` es imprescindible en APIs REST que autentican mediante JWT. Elimina la dependencia de sesiones HTTP en servidor.

Spring Security: Personalización del JwtAuthenticationConverter

El `JwtAuthenticationConverter` transforma los claims del token JWT en las autoridades (`GrantedAuthority`) que Spring Security usa internamente para decisiones de autorización. Es habitual que los roles vengan en claims personalizados.

```
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter converter =
        new JwtGrantedAuthoritiesConverter();
    // El claim "roles" contiene los roles en lugar del estándar "scope"
    converter.setAuthoritiesClaimName("roles");
    converter.setAuthorityPrefix("ROLE_");

    JwtAuthenticationConverter jwtConverter =
        new JwtAuthenticationConverter();
    jwtConverter.setJwtGrantedAuthoritiesConverter(converter);
    return jwtConverter;
}
```

❏ Si los roles provienen de un IdP externo como Keycloak, el path del claim puede ser anidado: `realm_access.roles`. En ese caso se necesita un converter personalizado que navegue la estructura JSON del JWT.

ASP.NET Core Identity: Arquitectura y extensibilidad

ASP.NET Core Identity proporciona un sistema completo de gestión de identidad con soporte para usuarios, roles, claims, tokens externos y almacenamiento persistente. Su arquitectura orientada a interfaces permite reemplazar cualquier componente.



UserManager<T>

API de alto nivel para crear, actualizar, eliminar usuarios y gestionar contraseñas, claims y roles asociados.



IUserStore<T>

Interfaz de persistencia. La implementación por defecto usa EF Core, pero puede sustituirse por cualquier ORM o base de datos NoSQL.



SignInManager<T>

Gestiona el proceso de inicio de sesión: validación de credenciales, 2FA, logout por intentos fallidos y cookies de autenticación.



RoleManager<T>

Gestión del ciclo de vida de roles y sus claims asociados, con soporte para políticas de autorización basadas en roles.

ASP.NET Core Identity: Configuración avanzada

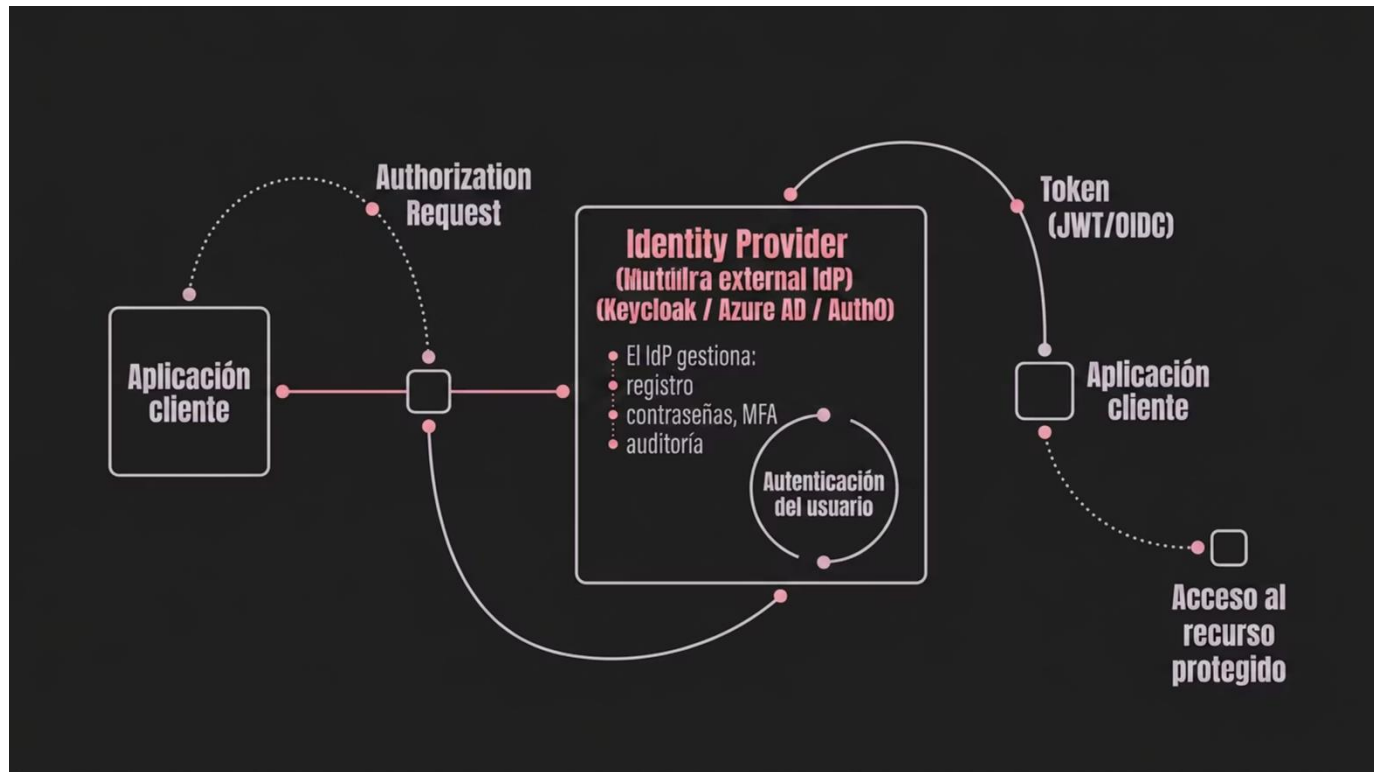
La configuración de Identity en el contenedor de dependencias permite ajustar las políticas de contraseñas, bloqueo de cuenta y validación de tokens según los requisitos de seguridad de cada entorno.

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options => {  
    // Política de contraseñas  
    options.Password.RequiredLength = 12;  
    options.Password.RequireNonAlphanumeric = true;  
    options.Password.RequireUppercase = true;  
  
    // Bloqueo de cuenta por intentos fallidos  
    options.Lockout.MaxFailedAccessAttempts = 5;  
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);  
    options.Lockout.AllowedForNewUsers = true;  
  
    // Confirmación de cuenta obligatoria  
    options.SignIn.RequireConfirmedEmail = true;  
})  
.AddEntityFrameworkStores<ApplicationDbContext>()  
.AddDefaultTokenProviders();
```

⚠ En entornos de producción, revisa siempre `RequireConfirmedEmail` y `MaxFailedAccessAttempts`. Los valores por defecto son demasiado permisivos para aplicaciones con datos sensibles.

Integración con proveedores externos de identidad (IdP)

La federación de identidad delega la autenticación a un proveedor externo de confianza (Keycloak, Azure AD, Auth0, Google...), eliminando la necesidad de gestionar credenciales directamente y reduciendo drásticamente la superficie de ataque.



El estándar OpenID Connect (OIDC), construido sobre OAuth2, es el protocolo recomendado para esta federación. El IdP emite un **ID Token** (identidad del usuario) y un **Access Token** (acceso a recursos), ambos en formato JWT.

Integración con Keycloak en Spring Boot

Keycloak es uno de los IdPs open-source más adoptados en entornos empresariales. La integración con Spring Boot se realiza configurando la aplicación como un **OAuth2 Resource Server** que valida los tokens emitidos por Keycloak.

```
# application.yml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          # URL del endpoint JWKS del realm de Keycloak
          jwk-set-uri: https://keycloak.example.com/realms/myrealm/protocol/openid-connect/certs
          # Validación del issuer del token
          issuer-uri: https://keycloak.example.com/realms/myrealm
```

 Spring Security descarga automáticamente las claves públicas del endpoint JWKS y las cachea. Cuando Keycloak rota las claves, el recurso server las renueva transparentemente sin necesidad de reinicio.

Integración con Azure AD en ASP.NET Core

Microsoft Identity Platform (Azure AD / Entra ID) se integra nativamente con ASP.NET Core mediante el paquete `Microsoft.Identity.Web`, que simplifica significativamente la configuración frente a la integración manual de OIDC.

```
// Program.cs
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApi(builder.Configuration.GetSection("AzureAd"));

// appsettings.json
{
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "your-tenant-id",
    "ClientId": "your-client-id",
    "Audience": "api://your-client-id"
  }
}
```

Validaciones automáticas

- Firma del token con claves públicas de Azure
- Expiración (exp) y emisión (iat)
- Audiencia (aud) y emisor (iss)

Claims disponibles

- `oid`: ID de objeto único del usuario
- `roles`: Roles de aplicación asignados
- `scp`: Scopes delegados

Gestión de usuarios y roles: modelo de autorización

La gestión de roles y permisos es uno de los aspectos más críticos del diseño de seguridad. Existen diferentes modelos según la complejidad y los requisitos del sistema.

RBAC — Role-Based Access Control

Los permisos se asignan a roles, y los roles a usuarios. Modelo simple y adecuado para la mayoría de aplicaciones empresariales. Ejemplo: `ROLE_ADMIN`, `ROLE_USER`.

ABAC — Attribute-Based Access Control

Los permisos se evalúan en función de atributos del usuario, del recurso y del contexto. Más expresivo que RBAC, pero más complejo de implementar y auditar.

Claims-Based Authorization

Las decisiones de autorización se basan en claims del token JWT. Permite granularidad fina sin necesidad de consultar una base de datos en cada petición.

Autorización basada en políticas (Spring & .NET)

Las políticas de autorización permiten encapsular reglas de acceso complejas de forma reutilizable, evitando lógica dispersa de seguridad en los controladores.

Spring Security — Method Security

```
@Configuration
@EnableMethodSecurity
public class MethodSecurityConfig {}

// En el servicio
@PreAuthorize(
    "hasRole('ADMIN') or " +
    "(hasRole('USER') and #userId == authentication.name)")
public UserDto getUser(String userId) { ... }

@PostAuthorize(
    "returnObject.ownerId == authentication.name")
public Document getDocument(Long id) { ... }
```

ASP.NET Core — Authorization Policies

```
builder.Services.AddAuthorization(options => {
    options.AddPolicy("SeniorDeveloper", policy =>
        policy.RequireRole("Developer")
            .RequireClaim("seniority", "senior")
            .RequireClaim("department", "engineering"));
});

// En el controlador
[Authorize(Policy = "SeniorDeveloper")]
[HttpGet("internal/architecture")]
public IActionResult GetArchitectureDocs() { ... }
```

BLOQUE 2

Flujos OAuth2

Authorization Code, Client Credentials y PKCE: cuándo y cómo usar cada uno



OAuth2: Repaso de roles y conceptos clave

Antes de analizar los flujos individuales, es fundamental tener claros los cuatro actores del modelo OAuth2 y sus responsabilidades. Confundir estos roles es una fuente habitual de errores de diseño.



Resource Owner

El usuario (o sistema) que posee los recursos protegidos y otorga el acceso.



Client

La aplicación que solicita acceso a los recursos en nombre del Resource Owner.



Authorization Server

Emite los tokens tras verificar la identidad del usuario y su consentimiento.

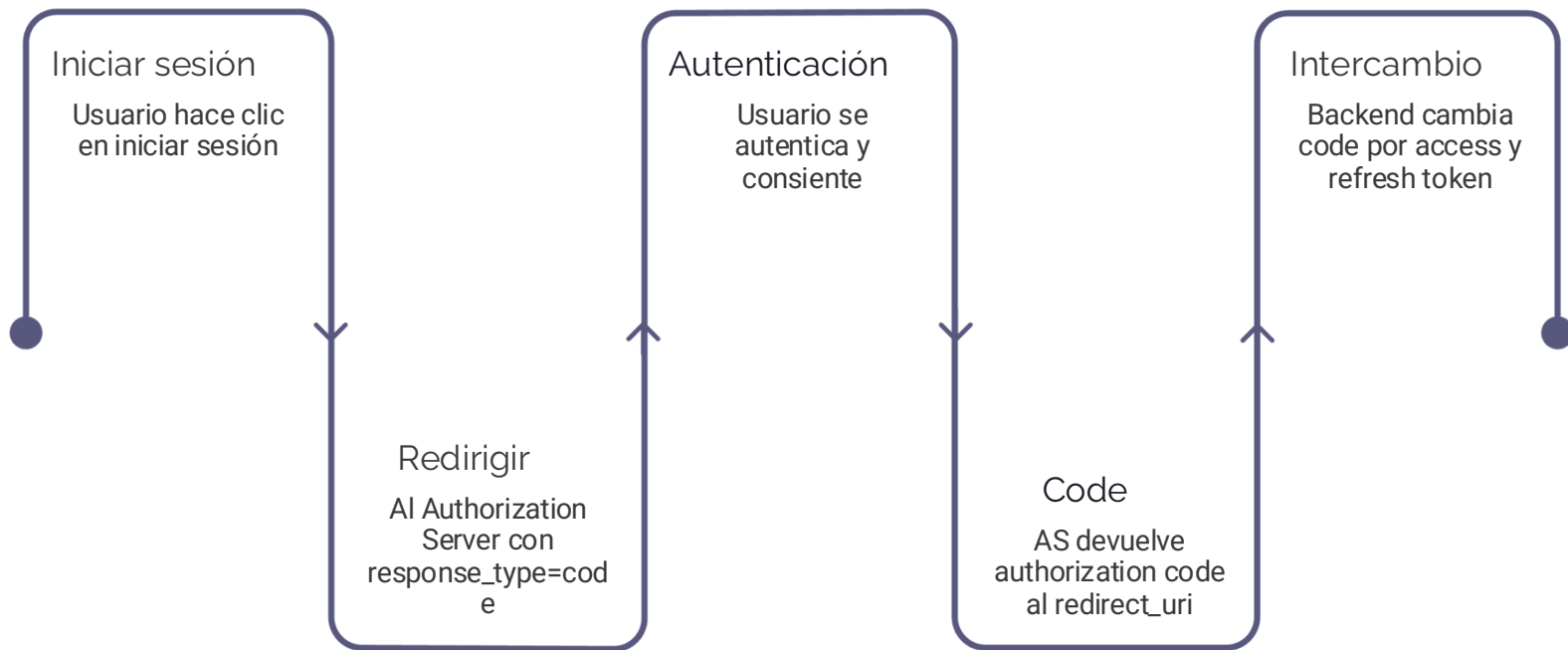


Resource Server

Expone los recursos protegidos y valida el Access Token en cada petición.

Flujo Authorization Code: El más seguro para usuarios

Es el flujo recomendado cuando hay un usuario humano involucrado. El Access Token nunca viaja por el navegador: se intercambia en el canal backend (back-channel), lo que lo protege frente a ataques de interceptación.



El **Authorization Code** tiene vida muy corta (segundos o pocos minutos) y es de un único uso. Esto limita la ventana de exposición en caso de interceptación. El intercambio por tokens se realiza desde el servidor, incluyendo el `client_secret`.

Authorization Code: Parámetros clave de la request

Conocer los parámetros exactos de cada fase del flujo es esencial tanto para implementar el cliente como para auditar las llamadas en producción.

Parámetro	Authorization Request	Token Request
<code>response_type</code>	<code>code</code>	—
<code>client_id</code>	✓	✓
<code>redirect_uri</code>	✓ (registrado)	✓ (debe coincidir)
<code>scope</code>	openid profile email	—
<code>state</code>	✓ (anti-CSRF)	—
<code>code</code>	—	✓ (recibido del AS)
<code>grant_type</code>	—	<code>authorization_code</code>
<code>client_secret</code>	—	✓ (confidential clients)



El parámetro `state` es obligatorio en implementaciones de producción. Su ausencia hace el flujo vulnerable a ataques CSRF que pueden forzar el inicio de sesión del usuario en una cuenta controlada por el atacante.

Flujo Client Credentials: Comunicación M2M

Este flujo está diseñado para escenarios **machine-to-machine** donde no hay usuario involucrado. Un servicio backend se autentica ante el Authorization Server usando su propia identidad (`client_id` + `client_secret`) para obtener un token de acceso.

Casos de uso típicos

- Comunicación entre microservicios
- Jobs programados que consumen APIs protegidas
- Integración con APIs de terceros (webhooks, pipelines CI/CD)
- Servicios sin contexto de usuario (procesamiento batch)

Flujo simplificado

```
POST /oauth/token
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=client_credentials
&client_id=service-a
&client_secret=secret
&scope=resource.read
```

```
// Respuesta:
{
  "access_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

Client Credentials: Gestión segura del `client_secret`

El `client_secret` es la credencial de identidad del servicio. Su exposición equivale a comprometer por completo la autenticación del microservicio. Debe tratarse con el mismo rigor que cualquier secreto crítico.

→ Nunca en código fuente ni repositorios

Usar variables de entorno, secretos de Kubernetes (`kubectl create secret`) o gestores de secretos como HashiCorp Vault o AWS Secrets Manager.

→ Rotación periódica automatizada

Implementar rotación automática del secret en los IdPs que lo soporten (Keycloak, Azure AD). Los secrets de larga duración aumentan el riesgo ante una filtración silenciosa.

→ Considerar mTLS como alternativa

Para entornos de alta seguridad, reemplazar el `client_secret` por autenticación mutua TLS (mTLS), eliminando el secreto compartido del flujo.

Flujo PKCE: Authorization Code para clientes públicos

Proof Key for Code Exchange (RFC 7636) es una extensión del flujo Authorization Code diseñada para clientes que no pueden mantener un `client_secret` de forma segura: SPAs, aplicaciones móviles y de escritorio.



PKCE garantiza que solo el cliente que inició el flujo puede canjear el código de autorización. Aunque el código sea interceptado, es inútil sin el `code_verifier` original, que nunca abandona el dispositivo del cliente.

PKCE: Implementación paso a paso

La implementación de PKCE requiere generar el par `code_verifier` / `code_challenge` en el cliente antes de iniciar el flujo. La mayoría de librerías OAuth2 modernas lo hacen automáticamente.

Generación del par (JavaScript/TypeScript)

```
// 1. Generar code_verifier (32-96 bytes aleatorios, Base64URL)
const array = new Uint8Array(32);
crypto.getRandomValues(array);
const codeVerifier = base64URLEncode(array);

// 2. Derivar code_challenge con SHA-256
const encoder = new TextEncoder();
const data = encoder.encode(codeVerifier);
const digest = await crypto.subtle.digest('SHA-256', data);
const codeChallenge = base64URLEncode(new Uint8Array(digest));

// 3. Guardar codeVerifier en sessionStorage para el paso del token
sessionStorage.setItem('pkce_verifier', codeVerifier);
```

Authorization Request con PKCE

```
GET /authorize?
  response_type=code
  &client_id=spa-client
  &redirect_uri=https://app.example.com/callback
  &scope=openid profile
  &state=RANDOM_STATE
  &code_challenge=BASE64URL(SHA256(verifier))
  &code_challenge_method=S256
```

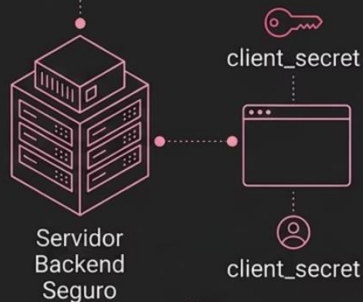
Token Request con PKCE

```
POST /token
  grant_type=authorization_code
  &code=AUTH_CODE
  &redirect_uri=https://app.example.com/callback
  &client_id=spa-client
  &code_verifier=ORIGINAL_VERIFIER
```

¿Qué flujo usar en cada escenario?

La elección del flujo OAuth2 incorrecto puede introducir vulnerabilidades graves. Esta decisión debe tomarse en la fase de diseño de la arquitectura, no en la implementación.

Authorization Code



Con backend seguro.
Casos: web apps tradicionales,
APIs con backend, apps
móviles con backend.

Authorization Code (con PKCE)



Sin backend seguro.
Casos: SPAs, apps móviles
nativas, Electron apps.

Client Credentials



Sin usuario.
Casos: microservicios, jobs
batch, integraciones
backend.

Implicit Flow y Password Grant: flujos a evitar

La especificación OAuth 2.1 deprecia explícitamente estos dos flujos. Es importante reconocerlos en sistemas legacy y planificar su migración.

Implicit Flow — Deprecado

El Access Token se devolvía directamente en el fragment de la URL de redirección, exponiéndolo en el historial del navegador, logs de servidor y referrer headers. **Reemplazar por Authorization Code + PKCE.**

Resource Owner Password Credentials — Deprecado

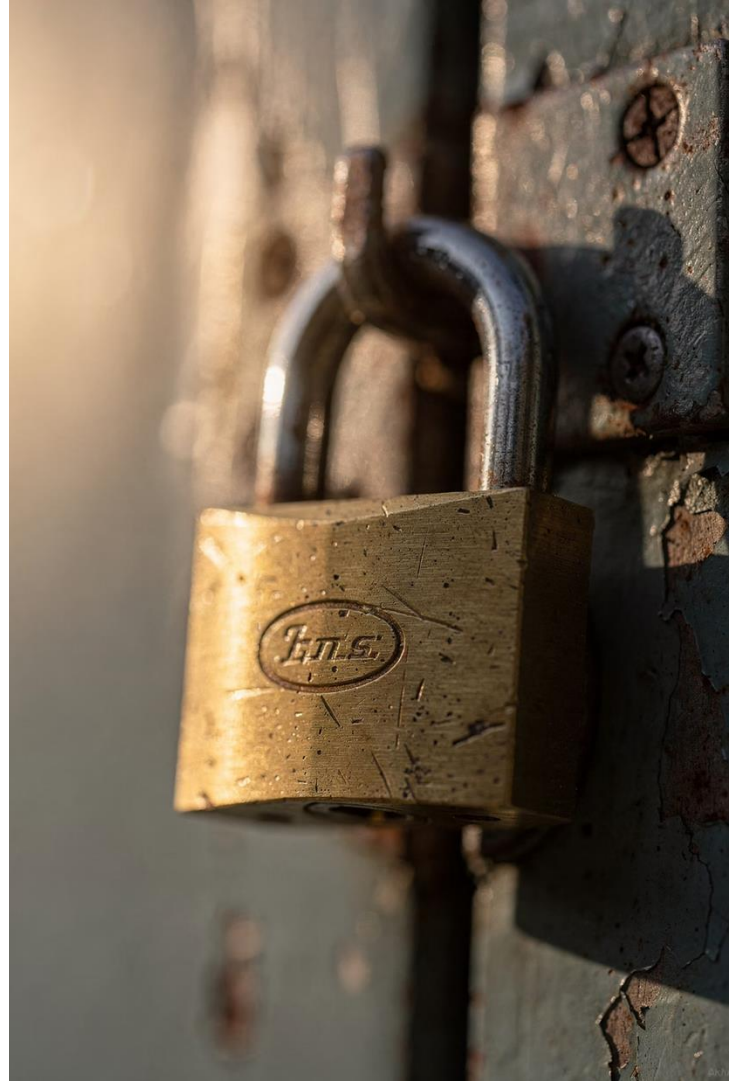
La aplicación cliente recibe directamente las credenciales del usuario (usuario/contraseña). Viola el principio de separación de credenciales y hace imposible implementar MFA o SSO correctamente. **Reemplazar por Authorization Code.**

⊗ Si encontráis estos flujos en sistemas existentes, documentadlos como deuda técnica de seguridad con prioridad alta. Su presencia activa es un hallazgo en cualquier auditoría de seguridad.

BLOQUE 3

Gestión de Tokens JWT

Firma y validación, expiración y renovación, buenas prácticas de almacenamiento



Anatomía de un JWT

Un JSON Web Token (RFC 7519) es una cadena compacta formada por tres partes codificadas en Base64URL y separadas por puntos. Cada parte tiene un rol específico en la verificación e interpretación del token.

Header	Payload (Claims)	Signature
Algoritmo de firma (<code>alg</code>) y tipo de token (<code>typ</code>). Ejemplo: <code>{"alg": "RS256", "typ": "JWT"}</code>	Datos del token: <code>sub</code> (sujeto), <code>iss</code> (emisor), <code>aud</code> (audiencia), <code>exp</code> (expiración), <code>iat</code> (emisión) y claims personalizados.	HMAC o RSA/ECDSA sobre Header+Payload. Garantiza integridad: cualquier modificación del payload invalida la firma.

 El payload de un JWT es solo codificado en Base64URL, **NO cifrado**. Cualquier persona con el token puede leer su contenido. Nunca incluir datos sensibles (contraseñas, PII innecesaria, secretos) en el payload.

Algoritmos de firma: HS256 vs RS256 vs ES256

La elección del algoritmo de firma determina el modelo de confianza del sistema. Esta decisión tiene implicaciones arquitectónicas importantes, especialmente en sistemas distribuidos.

Algoritmo	HS256 (HMAC)	RS256 (RSA)	ES256 (ECDSA)
Tipo	Simétrico	Asimétrico	Asimétrico
Clave	Secreta compartida	Par RSA 2048+ bits	Par EC P-256
Verificación	Requiere la clave secreta	Solo clave pública	Solo clave pública
Uso recomendado	Sistemas monolíticos	Arquitecturas distribuidas	Donde el tamaño importa
Riesgo principal	Compartir el secreto	Gestión del par de claves	Gestión del par de claves

 En microservicios, RS256 o ES256 son la elección correcta. El Resource Server puede verificar el token con la clave pública del JWKS endpoint sin necesidad de conocer la clave privada del Authorization Server.

Validación de JWT: El checklist completo

Validar un JWT no es solo verificar su firma. Un token con firma válida pero con claims incorrectos puede ser igualmente peligroso. Cada punto de este checklist debe implementarse explícitamente.

1 Verificar la firma criptográfica

Usando las claves del JWKS endpoint del emisor. Rechazar si no coincide o si el algoritmo es `none`.

2 Validar el claim `exp` (expiración)

El timestamp actual debe ser anterior al valor de `exp`. Considerar un margen de desfase de reloj (`clockSkew`) de máximo 5 minutos.

3 Validar el claim `iss` (emisor)

Debe coincidir exactamente con el Authorization Server esperado. Previene la aceptación de tokens de otros emisores.

4 Validar el claim `aud` (audiencia)

Debe incluir el identificador de nuestro Resource Server. Previene el uso de tokens emitidos para otro servicio (*audience confusion attack*).

Validación de JWT en código

Las librerías modernas implementan el checklist de validación de forma declarativa. Evitar implementaciones manuales de validación de JWT: son una fuente recurrente de vulnerabilidades graves.

Java — con JJWT

```
Jwts.parserBuilder()  
    .setSigningKeyResolver(jwksKeyResolver) // RS256  
    .requireIssuer("https://idp.example.com")  
    .requireAudience("api://my-service")  
    .setAllowedClockSkewSeconds(300)  
    .build()  
    .parseClaimsJws(token); // Lanza excepción si inválido
```

C# — con Microsoft.IdentityModel

```
var handler = new JwtSecurityTokenHandler();  
var result = handler.ValidateToken(token,  
    new TokenValidationParameters {  
        ValidIssuer = "https://idp.example.com",  
        ValidAudience = "api://my-service",  
        IssuerSigningKeys = jwks.Keys, // Claves del JWKS  
        endpoint  
        ValidateLifetime = true,  
        ClockSkew = TimeSpan.FromMinutes(5)  
    }, out SecurityToken validated);
```

⊗ Nunca usar `setSigningKey` con un algoritmo simétrico en un sistema distribuido. Si el Resource Server puede firmar tokens, el atacante que comprometa el servicio puede emitir tokens arbitrarios.

El ataque "alg:none" — Una vulnerabilidad clásica

Uno de los ataques más documentados contra JWT consiste en modificar el header para indicar `"alg": "none"`, haciendo que implementaciones vulnerables acepten tokens sin verificar la firma.

Cómo funciona el ataque

- El atacante obtiene un JWT válido (p. ej., como usuario normal)
- Modifica el payload (p. ej., cambia `role` a `admin`)
- Cambia el header a `{"alg": "none", "typ": "JWT"}`
- Elimina la firma del token
- La librería vulnerable acepta el token sin verificar

Mitigación obligatoria

- Configurar explícitamente los algoritmos permitidos en el parser
- Rechazar cualquier token con `alg:none`
- Usar librerías actualizadas (JJWT 0.11+, System.IdentityModel 6+)
- Nunca confiar en el algoritmo declarado en el header sin restricciones

⊗ Este ataque afectó a implementaciones de librerías populares (`jsonwebtoken < 4.2.2`, `pyjwt < 1.5.0`). Siempre mantener las dependencias actualizadas.

Expiración y tiempo de vida de los tokens

El tiempo de vida del Access Token es un equilibrio entre seguridad (tokens cortos limitan la ventana de ataque) y usabilidad (tokens muy cortos generan fricción). La estrategia de refresh permite conciliar ambos objetivos.

15m

Access Token

Tiempo de vida recomendado en producción.
Corto para limitar el impacto si es robado.

24h

Refresh Token (web)

Permite renovar el Access Token sin re-autenticar. Debe rotarse en cada uso.

30d

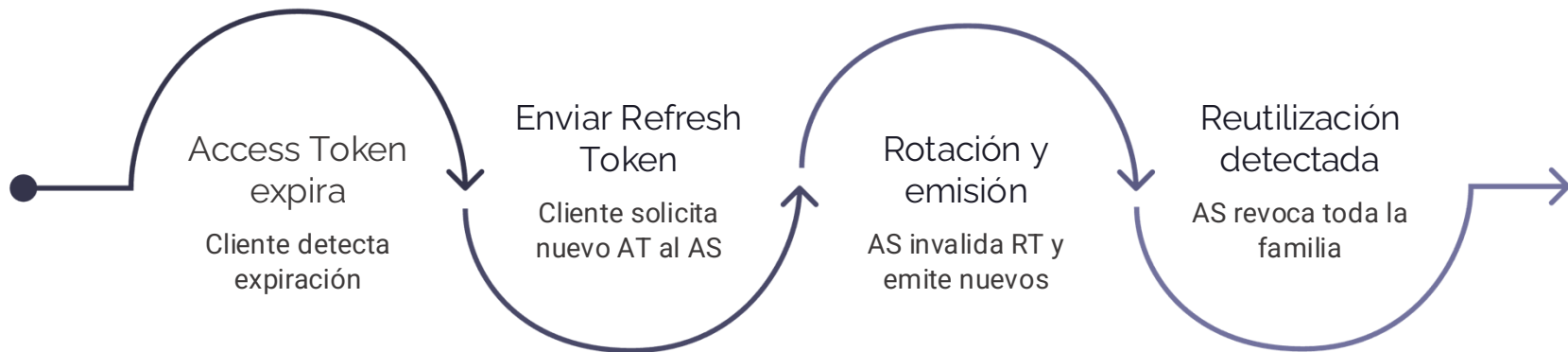
Refresh Token (móvil)

Mayor vida útil justificada por la experiencia de usuario, con rotación obligatoria.

Los valores anteriores son orientativos. El tiempo de vida óptimo depende de la sensibilidad de los datos expuestos y los requisitos de cumplimiento normativo (p. ej., PCI-DSS exige sesiones más cortas para operaciones de pago).

Renovación de tokens: Refresh Token Rotation

La rotación del Refresh Token es una medida de seguridad fundamental: cada vez que se usa un Refresh Token para obtener un nuevo Access Token, el Refresh Token antiguo se invalida y se emite uno nuevo. Esto permite detectar el uso de tokens robados.



- ✓ La detección de reutilización de Refresh Tokens (Refresh Token Reuse Detection) permite identificar sesiones comprometidas: si el RT antiguo es usado después de la rotación, significa que alguien más lo tiene. El AS debe revocar inmediatamente toda la familia de tokens de esa sesión.

Revocación de tokens: el problema de la naturaleza stateless

Los JWT son **stateless** por diseño: el Resource Server no necesita consultar ninguna base de datos para validarlos. Esto es muy eficiente, pero introduce una complicación: ¿cómo revocar un token antes de su expiración?

Token Introspection (RFC 7662)

El Resource Server consulta al Authorization Server si el token sigue siendo válido. Solución completa pero introduce latencia y dependencia en cada petición. Adecuada para operaciones sensibles.

Blacklist / Denylist

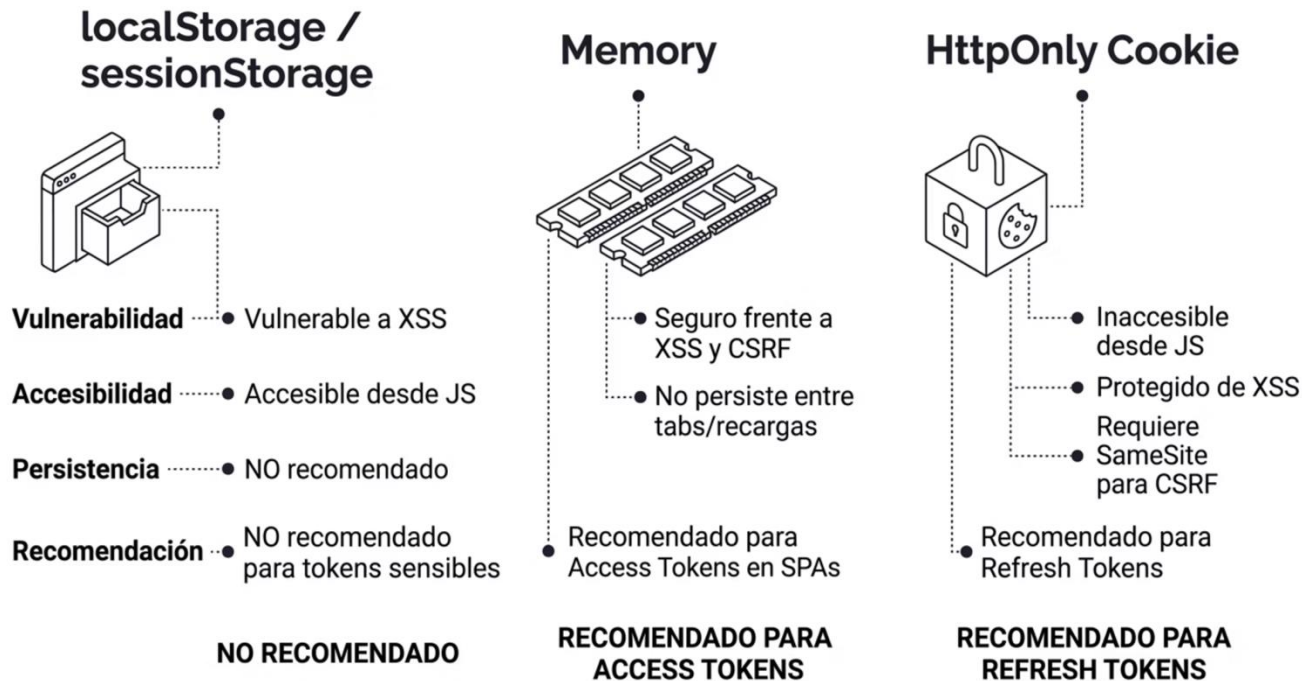
Almacenar los JTI (jti claim) de tokens revocados en Redis con TTL igual a la expiración del token. Eficiente y sin dependencia del AS, pero requiere consulta en caché.

Access Tokens de vida corta

Minimizar la necesidad de revocación manteniendo tiempos de vida muy cortos (1-15 minutos). La revocación del Refresh Token detiene la renovación de nuevos Access Tokens.

Buenas prácticas de almacenamiento de tokens

Dónde se almacena el Access Token y el Refresh Token es tan crítico como cómo se generan. Un token robusto almacenado de forma insegura puede ser comprometido con un ataque XSS básico.



Almacenamiento recomendado según tipo de aplicación

No existe una solución universal. La estrategia de almacenamiento debe adaptarse al tipo de cliente y al nivel de riesgo aceptable.

Tipo de app	Access Token	Refresh Token	Notas
SPA (React, Angular)	Memoria (variable JS)	HttpOnly Cookie	Proteger Cookie con SameSite=Strict
App móvil (iOS/Android)	Secure Storage del SO	Secure Storage del SO	Keychain (iOS), Keystore (Android)
Backend (servidor)	Memoria / Redis	Redis con TTL	Nunca en logs ni variables de entorno en texto plano
App de escritorio	Memoria en proceso	Credential Manager del SO	Evitar archivos de configuración en disco

Claims de seguridad recomendados en el payload

Diseñar el payload del JWT con los claims adecuados mejora tanto la seguridad como la capacidad de auditoría. Algunos claims son imprescindibles para una validación robusta.

Claims estándar (RFC 7519)

- `sub`: Identificador único e inmutable del usuario
- `iss`: URL del Authorization Server emisor
- `aud`: Identificador(es) del Resource Server
- `exp`: Timestamp de expiración (NumericDate)
- `iat`: Timestamp de emisión
- `jti`: ID único del token (para revocación y deduplicación)

Claims de contexto recomendados

- `roles / scope`: Permisos del usuario en este contexto
- `tenant_id`: En sistemas multi-tenant, imprescindible
- `session_id`: Para vincular el token a una sesión revocable
- `azp`: Cliente OAuth2 que solicitó el token (authorized party)



Minimizar los datos en el payload. Solo incluir lo necesario para autorización. Cualquier dato personal en el JWT implica consideraciones RGPD, ya que el token puede almacenarse en logs.

BLOQUE 4

Taller Práctico

Implementación de autenticación basada en tokens, validación de JWT y control de acceso



Estructura del taller

El taller se divide en tres ejercicios progresivos. Cada uno construye sobre el anterior, resultando en un sistema de autenticación completo con control de acceso granular al finalizar la sesión.



Ejercicio 1 — Configuración del Resource Server (20 min)

Configurar Spring Boot o ASP.NET Core para aceptar tokens JWT del IdP de laboratorio.
Verificar la validación de firma, issuer y audience.



Ejercicio 2 — Implementación del flujo PKCE (15 min)

Implementar un cliente que obtenga tokens mediante Authorization Code + PKCE e inyecte el Bearer Token en las llamadas a la API del ejercicio 1.



Ejercicio 3 — Control de acceso basado en roles y claims (15 min)

Proteger endpoints con roles específicos y validar claims personalizados del JWT.
Verificar el comportamiento ante tokens expirados, con audiencia incorrecta y con roles insuficientes.

Ejercicio 1: Configurar el Resource Server

El objetivo es arrancar un proyecto con la configuración mínima viable de Resource Server y verificar que rechaza peticiones sin token, con token inválido y acepta las correctas.

Spring Boot (Java)

```
# pom.xml – dependencia necesaria
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-
server</artifactId>
</dependency>

# application.yml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri:
http://localhost:8080/realms/lab/protocol/openid-connect/certs
          issuer-uri: http://localhost:8080/realms/lab
```

ASP.NET Core

```
// Program.cs
builder.Services
.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(options => {
  options.Authority = "http://localhost:8080/realms/lab";
  options.Audience = "lab-api";
  options.RequireHttpsMetadata = false; // Solo lab
});

builder.Services.AddAuthorization();

// Middleware (orden importa)
app.UseAuthentication();
app.UseAuthorization();
```

❏ `RequireHttpsMetadata = false` solo es aceptable en entornos de laboratorio. En producción, HTTPS es obligatorio y esta opción debe ser `true` (valor por defecto).

Ejercicio 1: Verificación de la configuración

Una vez configurado el Resource Server, debemos verificar que el comportamiento ante distintos tokens es el esperado. Estas pruebas son el mínimo aceptable antes de desplegar en cualquier entorno.

1

Sin token → 401 Unauthorized

`curl http://localhost:9090/api/hello` debe devolver 401. El endpoint está protegido.

2

Token válido → 200 OK

Obtener token de Keycloak con Client Credentials y adjuntarlo como `Authorization: Bearer <token>`.

3

Token expirado → 401 Unauthorized

Modificar el claim `exp` en el JWT (no la firma) y verificar que la validación falla.

4

Issuer incorrecto → 401 Unauthorized

Generar un token con un issuer diferente. Debe ser rechazado aunque la firma sea válida para ese issuer.

Ejercicio 2: Implementar el flujo PKCE

El objetivo es obtener un Access Token real usando Authorization Code + PKCE desde una aplicación cliente sencilla y utilizarlo para llamar a la API del ejercicio 1.

```
// Cliente Node.js con la librería 'openid-client'
import { Issuer, generators } from 'openid-client';

// 1. Descubrir la configuración del IdP via OIDC Discovery
const issuer = await Issuer.discover('http://localhost:8080/realms/lab');
const client = new issuer.Client({
  client_id: 'lab-pkce-client',
  redirect_uris: ['http://localhost:3000/callback'],
  response_types: ['code'],
  token_endpoint_auth_method: 'none' // Public client, sin secret
});

// 2. Generar code_verifier y code_challenge para PKCE
const codeVerifier = generators.codeVerifier();
const codeChallenge = generators.codeChallenge(codeVerifier);

// 3. Construir la Authorization URL
const authUrl = client.authorizationUrl({
  scope: 'openid profile',
  code_challenge: codeChallenge,
  code_challenge_method: 'S256',
  state: generators.state() // Protección anti-CSRF
});
```

Ejercicio 2: Completar el flujo PKCE — Token Exchange

Tras redirigir al usuario al Authorization Server y recibir el código de autorización en el callback, completamos el intercambio por tokens e invocamos la API protegida.

```
// 4. En el callback: intercambiar el code por tokens (incluye code_verifier)
app.get('/callback', async (req, res) => {
  const params = client.callbackParams(req);
  const tokenSet = await client.callback(
    'http://localhost:3000/callback',
    params,
    { code_verifier: codeVerifier, state: storedState }
  );

  console.log('Access Token:', tokenSet.access_token);
  console.log('Claims:', tokenSet.claims()); // Decodifica el ID Token

  // 5. Llamar a la API del Resource Server con el token obtenido
  const response = await fetch('http://localhost:9090/api/profile', {
    headers: {
      'Authorization': `Bearer ${tokenSet.access_token}`
    }
  });
  const data = await response.json();
  res.json(data);
});
```

✔ Si todo funciona correctamente, la API devuelve el perfil del usuario autenticado. El token ha pasado por la verificación de firma, issuer, audience y expiración en el Resource Server.

Ejercicio 3: Control de acceso basado en roles

Implementar endpoints con diferentes niveles de acceso y verificar que los tokens con los roles correctos obtienen acceso y los demás reciben el error apropiado.

Spring Security — Protección por roles

```
@RestController
@RequestMapping("/api")
public class LabController {

    // Acceso para cualquier usuario autenticado
    @GetMapping("/profile")
    public Map<String, Object> profile(JwtAuthenticationToken auth) {
        return auth.getToken().getClaims();
    }

    // Solo usuarios con rol DEVELOPER
    @GetMapping("/code-review")
    @PreAuthorize("hasRole('DEVELOPER')")
    public String codeReview() {
        return "Acceso a revisión de código";
    }

    // Solo ADMIN con claim de departamento específico
    @GetMapping("/admin/users")
    @PreAuthorize("hasRole('ADMIN') and #jwt.claims['dept']=='engineering'")
    public List<String> adminUsers(JwtAuthenticationToken jwt) {
        return List.of("user1", "user2");
    }
}
```

ASP.NET Core — Protección por políticas

```
// Definición de políticas
builder.Services.AddAuthorization(options => {
    options.AddPolicy("DeveloperOnly",
        p => p.RequireRole("developer"));

    options.AddPolicy("EngineeringAdmin", p => p
        .RequireRole("admin")
        .RequireClaim("dept", "engineering"));
});

// Aplicación en controladores
[Authorize(Policy = "DeveloperOnly")]
[HttpGet("code-review")]
public IActionResult CodeReview() => Ok("Acceso concedido");

[Authorize(Policy = "EngineeringAdmin")]
[HttpGet("admin/users")]
public IActionResult AdminUsers() => Ok(new[] {"user1", "user2"});
```

Ejercicio 3: Escenarios de prueba — Control de acceso

El taller finaliza verificando exhaustivamente el comportamiento del sistema ante diferentes combinaciones de token y endpoint. Este conjunto de pruebas debe formar parte de los tests de integración del proyecto.

Escenario	Endpoint	Esperado	Qué verificamos
Token válido, rol correcto	/api/code-review	200 OK	Flujo nominal correcto
Token válido, sin rol requerido	/api/code-review	403 Forbidden	Autorización granular
Token expirado	/api/profile	401 Unauthorized	Validación de expiración
Sin token (petición anónima)	/api/profile	401 Unauthorized	Protección del endpoint
Token de issuer diferente	/api/profile	401 Unauthorized	Validación de issuer
Token con audience incorrecta	/api/profile	401 Unauthorized	Validación de audience
Token manipulado (payload editado)	/api/code-review	401 Unauthorized	Verificación de firma

Depuración y herramientas de laboratorio

Conocer las herramientas adecuadas reduce significativamente el tiempo de depuración en el taller y en el trabajo diario con JWT y OAuth2.



jwt.io

Decodificador y verificador de JWT online. Permite inspeccionar header y payload, y verificar firmas con claves HMAC o RSA. Imprescindible para depuración.



Postman / Bruno / HTTPie

Cientes HTTP que soportan flujos OAuth2 automáticamente: obtienen el token y lo adjuntan a las peticiones sin intervención manual.



Keycloak Admin Console

Permite gestionar realms, clientes, usuarios y roles del laboratorio. Ofrece trazas de eventos de autenticación para analizar el flujo en tiempo real.



Spring Boot Actuator / .NET Diagnostics

Habilitar

`logging.level.org.springframework.security=TRACE` para ver el procesamiento interno de cada petición por la cadena de filtros.

Errores más frecuentes en el taller

Estos son los errores más habituales durante la implementación. Identificarlos rápidamente ahorra tiempo y refuerza la comprensión de los mecanismos internos.

401 cuando el token parece válido

Causas más frecuentes: `aud` del token no coincide con el audience configurado en el Resource Server; diferencia horaria entre máquinas superior al `clockSkew` configurado (5 min); HTTPS vs HTTP en el `iss`.

403 cuando el usuario tiene el rol correcto

El prefix de roles en Spring Security es `ROLE_` por defecto. Si el claim del JWT contiene `developer` pero se configura `hasRole('DEVELOPER')`, Spring busca `ROLE_DEVELOPER`. Verificar el `JwtAuthenticationConverter`.

Invalid_grant en el token exchange de PKCE

El `redirect_uri` en la Token Request no coincide exactamente con el de la Authorization Request (incluyendo trailing slash). También puede ser que el Authorization Code haya expirado o ya fue usado.

Patrones avanzados: Token Relay en microservicios

En arquitecturas de microservicios, un servicio que recibe una petición autenticada frecuentemente debe llamar a otros servicios en nombre del mismo usuario. El patrón **Token Relay** propaga el token original de forma transparente.



Spring Cloud Gateway

Filtro `TokenRelay` propaga automáticamente el Bearer Token del request entrante a todos los servicios downstream. Configuración de un línea en el YAML del gateway.

Consideración de seguridad

El Token Relay propaga los **permisos originales del usuario**. Para operaciones entre servicios sin contexto de usuario, usar Client Credentials con scopes específicos de servicio, no el token del usuario.

Patrón On-Behalf-Of (OBO) — Delegación de identidad

El flujo **On-Behalf-Of** (RFC 8693 Token Exchange) permite a un servicio intermedio obtener un nuevo token con el contexto de identidad del usuario original, pero con scopes adaptados al servicio downstream. Es más seguro que el Token Relay para casos sensibles.

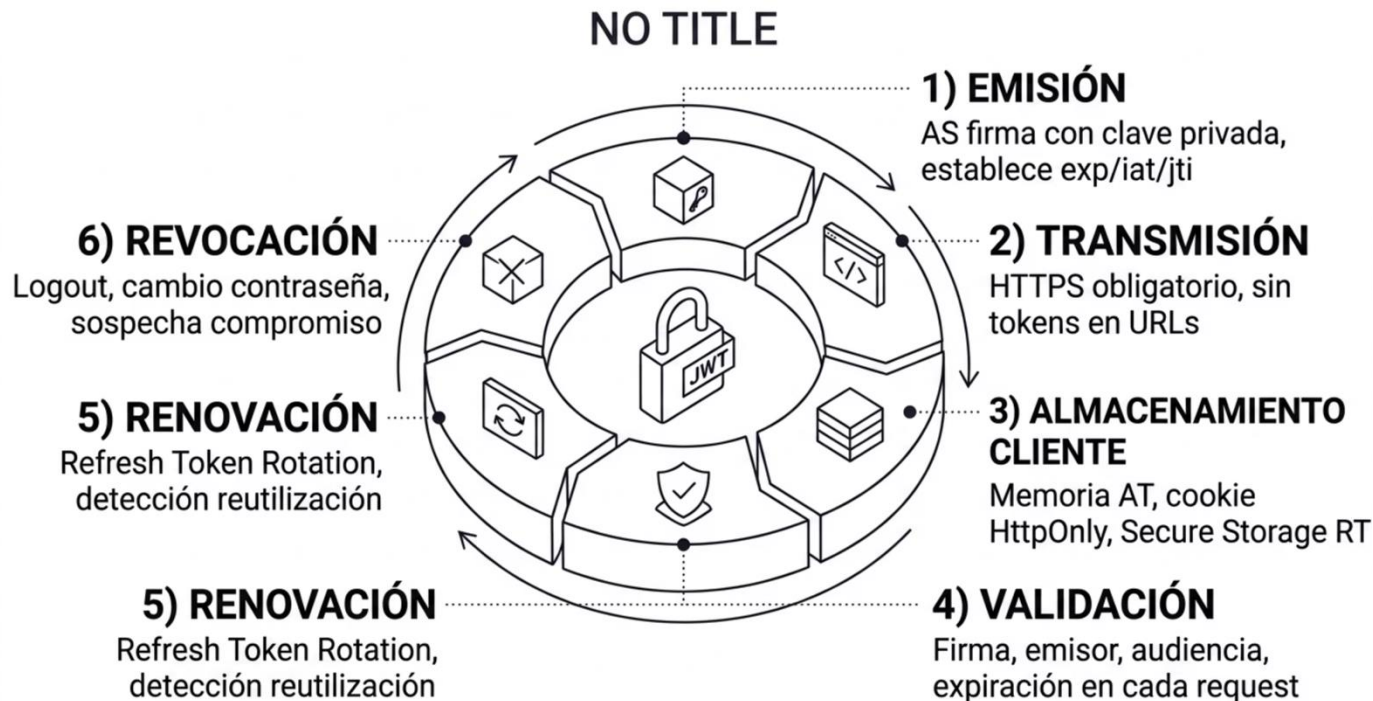
```
// Servicio A recibe un token de usuario y necesita llamar a Servicio B
// Servicio A solicita un token "en nombre del usuario" al Authorization Server
```

```
POST /oauth/token
grant_type=urn:ietf:params:oauth:grant-type:token-exchange
&subject_token=ACCESS_TOKEN_USUARIO
&subject_token_type=urn:ietf:params:oauth:token-type:access_token
&requested_token_type=urn:ietf:params:oauth:token-type:access_token
&audience=service-b
&scope=service-b:read
&client_id=service-a
&client_secret=service-a-secret
```

❗ El nuevo token emitido tiene el **sub** del usuario original pero la **aud** del Servicio B y los scopes limitados. El Servicio B sabe que la petición viene de Servicio A actuando en nombre del usuario gracias al claim **act** (actor).

Seguridad en el ciclo de vida del token: visión completa

Una implementación robusta de gestión de tokens debe cubrir todo el ciclo de vida, desde la emisión hasta la revocación, sin dejar huecos que puedan ser explotados.



Lista de comprobación de seguridad — Tokens

Esta lista debe usarse como referencia en revisiones de código y auditorías internas. Cada punto aborda una vulnerabilidad real documentada en los informes de seguridad de aplicaciones web.

Emisión y validación

- ✓ Usar RS256 o ES256, nunca HS256 en sistemas distribuidos
- ✓ Validar firma, issuer, audience y expiración en cada request
- ✓ Rechazar algoritmo `none` explícitamente
- ✓ Incluir `jti` para deduplicación y revocación
- ✓ Tiempo de vida del AT $\leq$ 15 minutos en producción

Almacenamiento y transmisión

- ☐ Tokens solo sobre HTTPS (incluido entorno de staging)
- ☐ AT en memoria; RT en HttpOnly Cookie o Secure Storage
- ☐ Nunca tokens en URLs, query params o logs de aplicación
- ☐ Refresh Token Rotation habilitado
- ☐ Revocación implementada (blacklist o introspección)

Consideraciones para entornos multi-tenant

Los sistemas SaaS con múltiples inquilinos (tenants) añaden una dimensión adicional de seguridad. Una mala implementación puede permitir que un usuario de un tenant acceda a los recursos de otro.

Claim `tenant_id` en el JWT

Incluir el identificador del tenant en el payload del token y validarlo en cada operación. Nunca confiar en el `tenant_id` del request body o de la URL sin cruzarlo con el del token.

Realm separado por tenant en Keycloak

La estrategia de un realm por tenant en Keycloak proporciona aislamiento completo de usuarios, roles y configuración, a costa de mayor complejidad operativa.

Row-level security en base de datos

Combinar la validación del `tenant_id` del JWT con filtros automáticos en el ORM (Hibernate Filters, EF Core Query Filters) que añadan la condición de tenant a todas las consultas.

Spring Security: Expresiones SpEL avanzadas

Spring Security Expression Language (SpEL) permite crear condiciones de autorización muy expresivas que van más allá de la simple comprobación de roles, accediendo al contexto de la petición, los argumentos del método y los claims del JWT.

```
@Service
public class DocumentService {

    // Acceso al claim personalizado 'tenant_id' del JWT
    @PreAuthorize("authentication.token.claims['tenant_id'] == #tenantId")
    public List<Document> getDocuments(String tenantId) { ... }

    // Combinación de rol y propiedad del objeto retornado
    @PostAuthorize("hasRole('VIEWER') and returnObject.ownerId == authentication.name")
    public Document getDocument(Long docId) { ... }

    // Acceso a beans de Spring para lógica de autorización compleja
    @PreAuthorize("@documentSecurityService.canEdit(authentication, #docId)")
    public void updateDocument(Long docId, DocumentDto dto) { ... }
}

// Bean de seguridad reutilizable
@Component("documentSecurityService")
public class DocumentSecurityService {
    public boolean canEdit(Authentication auth, Long docId) {
        // Lógica de negocio para determinar si el usuario puede editar
        return documentRepo.findById(docId)
            .map(doc -> doc.getOwnerId().equals(auth.getName())) ||
            auth.getAuthorities().contains(ADMIN_AUTHORITY))
            .orElse(false);
    }
}
```

OpenID Connect: el ID Token y el UserInfo endpoint

OAuth2 es un framework de **autorización**. OpenID Connect añade la capa de **autenticación** sobre OAuth2 mediante el ID Token y el endpoint estándar UserInfo, permitiendo conocer quién es el usuario que se ha autenticado.

ID Token

JWT emitido junto al Access Token cuando el scope incluye `openid`. Contiene la identidad verificada del usuario: `sub`, `name`, `email`, `email_verified`, y otros claims estándar OIDC.

Access Token

Para autorización de acceso a recursos. El Resource Server lo valida en cada petición. Su contenido es opaco para el cliente por diseño.

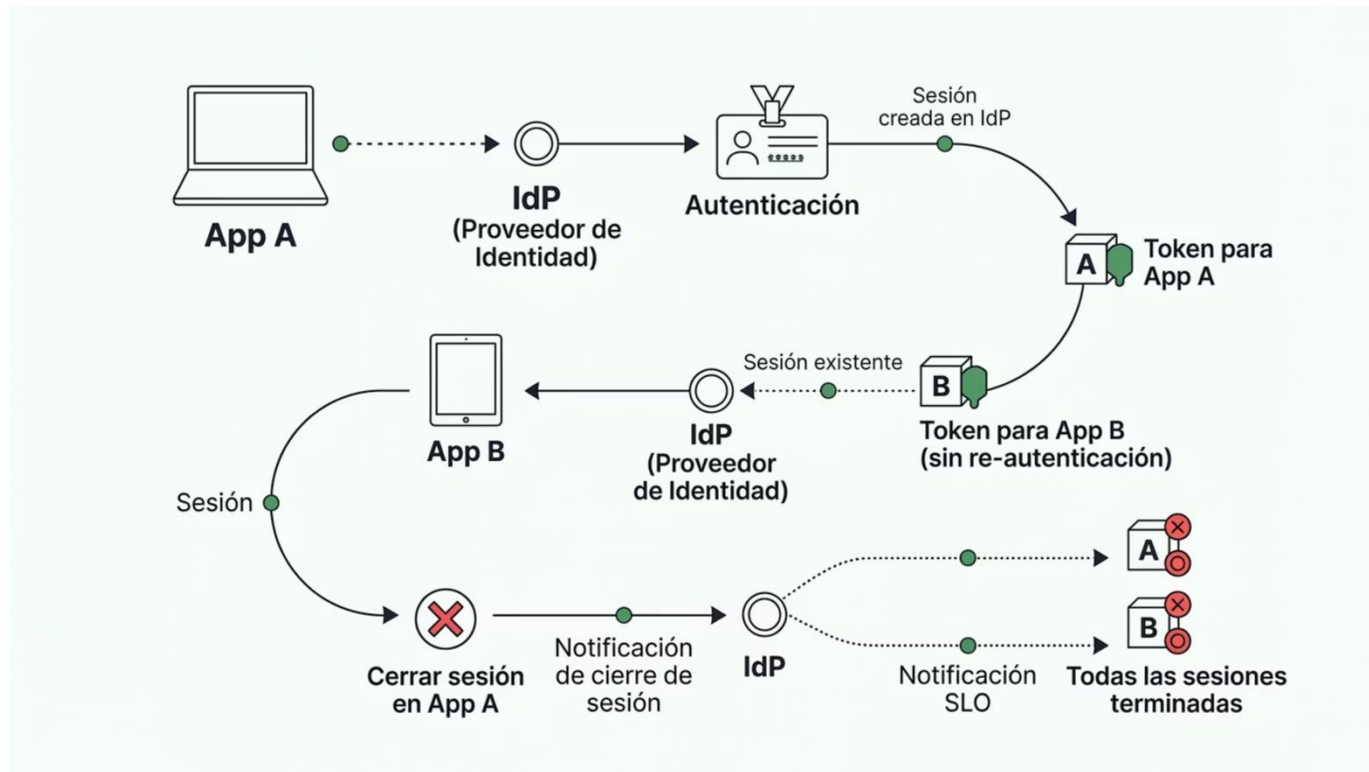
UserInfo Endpoint

Endpoint estándar (`/userinfo`) del AS que devuelve información del usuario autenticado cuando se presenta un Access Token válido con scope `openid`.

- ❏ El ID Token debe ser validado por el cliente (firma, nonce, aud, iss, exp). El Access Token no debe ser inspeccionado por el cliente —solo por el Resource Server—. Esta separación es fundamental en el modelo OIDC.

Single Sign-On (SSO) y Single Logout (SLO)

OpenID Connect habilita el SSO de forma nativa: una única autenticación en el IdP da acceso a múltiples aplicaciones del mismo dominio de seguridad. La sesión del IdP actúa como fuente de verdad.



El **Single Logout (SLO)** es más complejo que el SSO. Requiere que el IdP notifique a todas las aplicaciones con sesión activa. Implementar SLO correctamente es crítico: un logout parcial deja ventanas de acceso no autorizado si el usuario abandona una máquina compartida.

Resumen: Decisiones de diseño clave

Tras las tres horas de sesión, estas son las decisiones arquitectónicas que deben tomarse conscientemente en cualquier sistema de autenticación moderno. No existe una respuesta universal: el contexto determina la elección óptima.

Decisión	Opciones	Criterio de elección
Framework de autenticación	Spring Security / ASP.NET Identity	Stack tecnológico del proyecto
Flujo OAuth2	Auth Code + PKCE / Client Credentials	¿Hay usuario? → PKCE. ¿M2M? → CC
Algoritmo de firma JWT	HS256 / RS256 / ES256	Monolito → HS256. Distribuido → RS256/ES256
Almacenamiento del AT	Memoria / HttpOnly Cookie	SPA → Memoria. SSR → Cookie
Revocación de tokens	Vida corta / Blacklist / Introspección	Latencia vs. seguridad inmediata
IdP	Keycloak / Azure AD / Auth0	On-premise vs. cloud vs. coste operativo

Vulnerabilidades frecuentes relacionadas con autenticación

Las vulnerabilidades de autenticación aparecen consistentemente en el OWASP Top 10 (A07: Identification and Authentication Failures). Conocer los patrones de ataque es tan importante como conocer la defensa.

JWT Algorithm Confusion

Cambio del algoritmo de RS256 a HS256 usando la clave pública como secret HMAC. Mitigación: validar el alg contra una whitelist.

Open Redirect en OAuth2

Un redirect_uri no validado correctamente permite redirigir el Authorization Code a un servidor del atacante. Mitigación: validación exacta del redirect_uri registrado.

CSRF en el flujo OAuth2

Ausencia del parámetro state permite forzar el login de un usuario en una cuenta controlada por el atacante. Mitigación: state obligatorio y vinculado a la sesión.

Token Leakage en logs

Los Access Tokens registrados en logs de aplicación o de infraestructura (Nginx, ALB) quedan expuestos. Mitigación: filtrar cabeceras Authorization en todos los sistemas de logging.

Recursos y referencias

Material de referencia recomendado para profundizar en los temas de esta sesión y mantenerse al día con las especificaciones en evolución.

Especificaciones y RFCs

- [RFC 6749 – OAuth 2.0 Authorization Framework](#)
- [RFC 7519 – JSON Web Token \(JWT\)](#)
- [RFC 7636 – PKCE for OAuth Public Clients](#)
- [RFC 8693 – OAuth 2.0 Token Exchange](#)
- [OpenID Connect Core 1.0 \(openid.net\)](#)
- [OAuth 2.1 Draft \(auth0.com/docs/authenticate\)](#)

Documentación oficial

- [Spring Security Reference – docs.spring.io/spring-security](#)
- [ASP.NET Core Security – learn.microsoft.com](#)
- [Keycloak Server Administration Guide](#)
- [OWASP Authentication Cheat Sheet](#)
- [jwt.io – Debugger y librerías por lenguaje](#)
- [PortSwigger Web Security Academy – OAuth labs](#)

Próxima sesión: Seguridad en APIs

En la Sesión 3 abordaremos la seguridad a nivel de API: cómo proteger los endpoints expuestos de forma completa, desde la capa de red hasta la validación de entrada, pasando por las vulnerabilidades más comunes.



Rate Limiting y Throttling

Protección contra abuso, scraping y ataques de fuerza bruta a nivel de API.



CORS y Headers de seguridad

Configuración correcta de CORS, HSTS, CSP y otros headers HTTP de seguridad.



OWASP Top 10 Backend

SQL Injection, XSS, CSRF, XXE y otras vulnerabilidades comunes en APIs REST.



Gestión de secretos

HashiCorp Vault, AWS Secrets Manager y Azure Key Vault en entornos productivos.



Sesión 2 — Conclusiones

Habéis implementado hoy un sistema de autenticación completo: desde la configuración avanzada del framework hasta el ciclo de vida completo del token. Lleváis una base sólida para diseñar sistemas seguros por defecto.



Usa el framework, no reinventes

Spring Security y ASP.NET Identity resuelven correctamente problemas complejos. La configuración personalizada es donde aparecen los errores.



PKCE para usuarios, CC para servicios

La elección del flujo OAuth2 correcto es una decisión arquitectónica que debe tomarse en el diseño, no en la implementación.



Tokens cortos + rotación = defensa en profundidad

Un token de 15 minutos con rotación de refresh token limita drásticamente el impacto de cualquier filtración.